Azərbaycan Respublikası Elm və Təhsil Nazirliyi

Qarabağ Universiteti
Mühəndislik Fakültəsi

Verilənlərin Strukturu və Alqoritmlər

Mexatronika və robototexnika mühəndisliyi və Kompüter mühəndisliyi ixtisası üzrə bakalavr proqramının tələbələri üçün.

Laboratoriya işi № 2

Tələbənin adı və soyadı: Səbinə Alıyeva
Qrup: KOM25A
Müəllim: Şamil Hümbətov
Tarix: 19 oktyabr 2026

2026

Mövzu: Linked List, Stack və Queue Verilənlər Strukturlarının Tətbiqi, Təhlili və Müqayisəsi

Əsas mənbə: *Mark Allen Weiss, Data Structures and Algorithm Analysis in C++ (4th Edition)*

Məqsəd

Bu laboratoriya işinin məqsədi tələbələrin Linked List, Stack və Queue verilənlər strukturlarının nəzəri əsaslarını öyrənməsi, onları sıfırdan C++ / Python dilində implementasiya etməsi, eksperimental sınaqlar aparması və nəticələri elmi əsaslarla təhlil etməsidir.

Tapşırıq 1

Smart Music Playlist (Linked List)

A. Araşdırma

1. Array və Linked List arasında əsas fərqlər hansılardır?

   Massivdə elementlər yaddaşda bir-birinin ardınca, vahid blok halında düzülür: indekslə müraciət O(1) vaxtında baş verir, lakin ortadan element əlavəsi və silinməsi qalan elementlərin sürüşdürülməsini tələb etdiyindən O(n) xərci ilə nəticələnir. Linked List-də isə düyünlər yaddaşda sərbəst paylanır və yalnız göstəricilərlə bir-birinə bağlanır: müəyyən mövqeyə əlavə və silmə göstəricilərin yenidən yönəldilməsi hesabına aparılır, eyni zamanda istənilən elementə müraciət üçün siyahını gəzmək (O(n)) lazımdır.

2. Niyə musiqi playlistləri üçün Linked List uyğun seçim hesab oluna bilər?

   Playlistdə mahnılar daim əlavə olunur və silinir, onların dəqiq sayı isə əvvəlcədən məlum deyil. Dinləmə ardıcıl şəkildə aparılır və strukturun ölçüsü dinamik olaraq dəyişməlidir. Bu şərtlər Linked List-in təbiətinə —sərbəst böyüməyə və sürüşdürmə olmadan əlavə-silməyə — daha uyğun gəlir.

3. Singly və Doubly Linked List arasında hansı üstünlüklər və çatışmazlıqlar mövcuddur?

   Singly Linked List hər düyündə cəmi bir göstərici saxladığında az yaddaş istifadə edir, ancaq yalnız irəli istiqamətdə hərəkət mümkündür. Doubly Linked List iki göstərici sayəsində hər iki istiqamətdə keçid verir və verilmiş düyünü əvvəlki düyünü axtarmadan O(1) vaxtında silməyə imkan yaradır; bunun müqabilində hər düyün üçün əlavə göstərici yaddaşı tələb olunur.

4. Linked List-də axtarış niyə Array-dən daha yavaş ola bilər?

   Linked List-də axtarış yavaş nəticə verir, çünki düyünlər yaddaşda ardıcıl yerləşmir, indekslə birbaşa müraciət mövcud deyil və prosessor keşindən səmərəli istifadə olunmur. Nəticə etibarilə axtarış siyahının başdan sona gəzilməsini, yəni O(n) vaxtını tələb edir.

5. Playlistdə mahnıların sırasını dəyişmək üçün hansı struktur daha əlverişlidir?

Mahnıların sırasını dəyişmək və əvvəlki mahnıya qayıtmaq kimi əməliyyatlar üçün Doubly Linked List daha əlverişlidir: iki istiqamətli keçid yer dəyişməni sadələşdirir və geriyə naviqasiyanı asanlaşdırır.

B. Proqramlaşdırma(Kod)

```
class SongNode:
    def __init__(self, song_id, name, artist, duration):
        self.song_id = song_id
        self.name = name
        self.artist = artist
        self.duration = duration
        self.next = None
class Playlist:
    def __init__(self):
        self.head = None
        self.tail = None
        self.size = 0
        self.current = None
    def addSong(self, song_id, name, artist, duration):
        node = SongNode(song_id, name, artist, duration)
        if self.size == 0:
            self.head = node
            self.tail = node
        else:
            self.tail.next = node
            self.tail = node
        self.size = self.size + 1
        return node
```

```
 def searchSong(self, song_id):
        node = self.head
        while node is not None:
            if node.song_id == song_id:
                break
            node = node.next
        return node
    def deleteSong(self, song_id):
        prev = None
        cur = self.head
        found = False
        while cur is not None:
            if cur.song_id == song_id:
                found = True
                break
            prev = cur
            cur = cur.next
        if not found:
            return False
        if prev is None:
            self.head = cur.next
        else:
            prev.next = cur.next
        if cur is self.tail:
            self.tail = prev
        if cur is self.current:
```

```
            self.current = prev
        self.size = self.size - 1
        return True
    def displayPlaylist(self):
        if self.size == 0:
            print("Playlist boşdur.")
            return
        parts = []
        node = self.head
        while node is not None:
            parts.append(f"[{node.song_id}] {node.name} - {node.artist} ({node.duration})")
            node = node.next
        parts.append("None")
        print(" -> ".join(parts))
    def countSongs(self):
        return self.size
    def playNextSong(self):
        if self.size == 0:
            print("Playlist boşdur.")
            return None
        if self.current is None or self.current.next is None:
            self.current = self.head
        else:
            self.current = self.current.next
        print(f"Oynatılır: {self.current.name} - {self.current.artist} ({self.current.duration})")
        return self.current
```

```
def main():
    pl = Playlist()
    choice = ""
    while choice != "7":
        print("\n1. mahnı əlavə et\n2. mahnı sil\n3. mahnı axtar\n4. Playlisti göstər\n5. Növbəti mahnını oynat\n6. Mahnıları say\n7. Çıxış")
        choice = input("seçim: ").strip()
        if choice == "1":
            sid = int(input("ID: "))
            if pl.searchSong(sid):
                print("mahnı artıq mövcuddur.")
                continue
            name = input("Mahnı adı: ")
            artist = input("Artist: ")
            duration = input("Mahnı müddəti: ")
            pl.addSong(sid, name, artist, duration)
            print("mahnı əlavə olundu.")
        elif choice == "2":
            sid = int(input("ID: "))
            if pl.deleteSong(sid):
                print("mahnı silindi.")
            else:
                print("mahnı tapılmadı.")
        elif choice == "3":
            s = pl.searchSong(int(input("ID: ")))
            if s is None:
```

```
            print("mahnı tapılmadı.")
        else:
            print(f"Tapıldı: {s.name} - {s.artist} ({s.duration})")
    elif choice == "4":
        pl.displayPlaylist()
    elif choice == "5":
        pl.playNextSong()
    elif choice == "6":
        print("Mahnılar:", pl.countSongs())
    elif choice != "7":
        print("yoxdur.")
if __name__ == "__main__":
    main()
```

C. Təcrübə və Eksperiment

Insert üçün N əlavə əməliyyatının ümumi vaxtı, Search və Delete üçün isə ən pis halda — siyahının sonuncu elementi üzərində — bir əməliyyatın orta vaxtı ölçülüb. Hər ölçmə 5 dəfə təkrarlanaraq median dəyəri götürülüb (time.perf_counter). Nəticələr Cədvəl 1 və Şəkil 1-də təqdim olunur.

D. Analiz və Nəticələrin Şərhi

1. Element sayı artdıqca hansı əməliyyat daha çox təsirləndi?

   Search və Delete. n = 100-dən 10 000-ə artdıqda Insert təxminən 115 dəfə (0.0054 → 0.617 ms), Search 22 700 dəfə (0.0084 → 191.5 ms), Delete 21 600 dəfə (0.0097 → 209.7 ms) artdı.

2. Təcrübə nəticələri Big-O nəzəriyyəsini təsdiqləyirmi?

   Bəli. Insert n-dən asılı olmayaraq sabit qaldı (~55 ns), yəni O(1). Search və Delete bir əməliyyatın müddəti n ilə mütənasib artdı (84 ns → 19 µs), yəni O(n). Kiçik sapma cache effekti ilə izah olunur.

3. Eyni problemi Array istifadə etməklə həll etsəydiniz nəticələr necə olardı?

   Sona Insert O(1) amortizasiya olardı və cache sayəsində daha sürətli işləyərdi. Search yenə O(n) olardı (sıralıdırsa binary search ilə O(log n)). Delete-də elementlərin sürüşdürülməsi lazım olduğu üçün O(n) olardı.

4. Linked List-in əsas üstünlüyü nədir?

   Ölçüsü dinamikdir və mövqe məlum olduqda əlavə və silmə elementləri sürüşdürmədən, yalnız göstəriciləri dəyişməklə O(1)-də yerinə yetirilir.

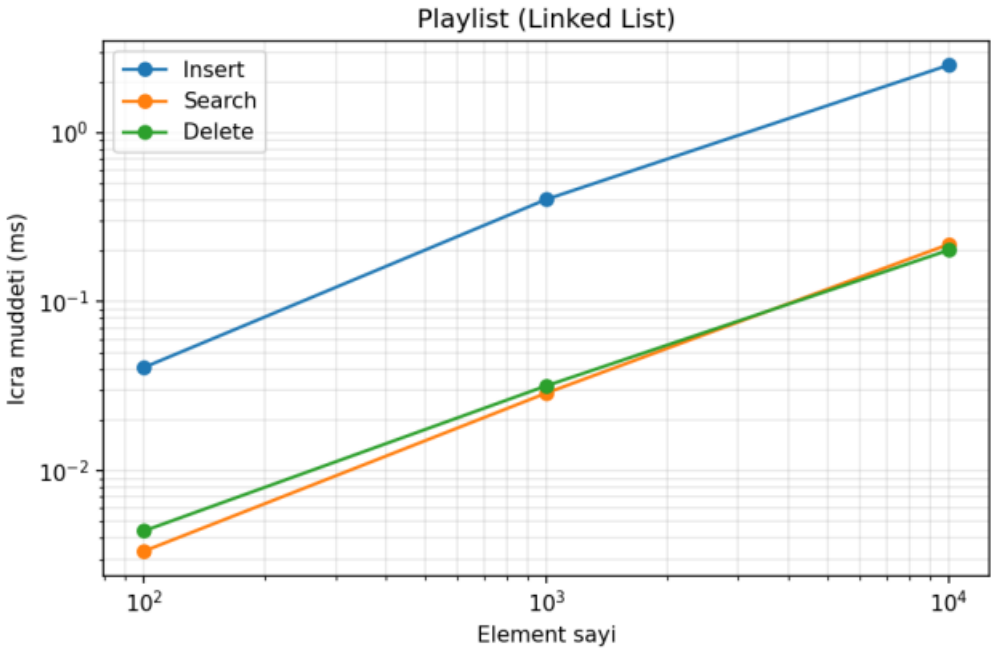
5. Playlist tətbiqində Doubly Linked List istifadə etmək daha məntiqli olardımı?

   Bəli. prev göstəricisi sayəsində əvvəlki mahnıya keçid O(1)-də olur və silmə sadələşir. Əlavə yaddaş xərci hər node üçün cəmi bir göstəricidir.

Cədvəl

Element sayı	Insert (ms)	Search (ms)	Delete (ms)
100	0.0408	0.003338	0.0044
1000	0.4033	0.028794	0.0318

Qrafik



Tapşırıq 2

Browser History Manager (Stack)

A. Araşdırma

1. Stack strukturunda LIFO prinsipi nədir?

   Stack LIFO (Last In — First Out) prinsipi ilə işləyir: strukturə ən son daxil edilən element ilk növbədə çıxarılır.

2. Browser History sistemi nə üçün Stack istifadə edir?

   «Back» düyməsi istifadəçini ən son baxılan səhifədən bir əvvəlkinə qaytarır ki, bu da məhz pop əməliyyatının icrası deməkdir — ona görə tarixçənin saxlanması üçün Stack uyğundur.

3. Undo əməliyyatı necə Stack vasitəsilə həyata keçirilir?

   Hər yeni əməliyyat stack-ə push edilir; Undo çağırıldıqda isə top-dakı element pop olunaraq istifadəçiyə geri qaytarılır

4. Queue istifadə olunsaydı hansı problemlər yaranardı?

   Queue istifadə olunsaydı, geri qayıdış ən köhnə səhifəyə doğru baş verərdi ki, bu da Undo məntiqini tamamilə pozardı.

5. Linked List və Array əsaslı Stack implementasiyalarını müqayisə edin.

   Linked List əsaslı stack-də hər element üçün ayrıca düyün yaradılır və əlavə yaddaş sərf olunur, strukturun ölçüsü isə məhdudlaşdırılmır. Array əsaslı stack daha kompakt yaddaş bölgüsünə və keşdən daha yaxşı istifadəyə malikdir, lakin tutum dolduqda yenidən yer ayırma xərci yaranır.

B. Proqramlaşdırma

```
from datetime import datetime

 class ActionNode:

def __init__(self, action_id, name):

   self.action_id = action_id

   self.name = name

   self.time = datetime.now().strftime("%H:%M:%S")

   self.next = None

 class UndoStack:

def __init__(self):

   self.top = None

   self.size = 0

 def pushAction(self, action_id, name):

   node = ActionNode(action_id, name)

   self.top, node.next = node, self.top
```

```
  self.size = self.size + 1
  return node
 def undoAction(self):
  if self.size == 0:
     return None
  node = self.top
  self.top = node.next
  node.next = None
  self.size = self.size - 1
  return node
def currentAction(self):
  return self.top
 def displayHistory(self):
  if self.size == 0:
     print("Tarixçə boşdur.")
     return
  lines = []
  node = self.top
  while node is not None:
     lines.append(f"  [{node.action_id}] {node.name} ({node.time})")
     node = node.next
  print("Tam Tarixçə:")
  print("\n".join(lines))
def countActions(self):
  return self.size
def main():
```

```
st = UndoStack()
next_id = 1
while True:
    print("\n1. Əməl yerinə yetir\n2. Ləğv et\n3. Cari Əməl\n4. Tam Tarixçə\n5. Çıxış")
    choice = input("seçim: ").strip()
    if choice == "1":
        name = input("Əməl adı: ")
        st.pushAction(next_id, name)
        next_id += 1
        print("Əməl yerinə yetirildi.")
    elif choice == "2":
        removed = st.undoAction()
        if removed is None:
            print("Ləğv etmək üçün heç bir əməl yoxdur.")
        else:
            print("Ləğv olunur: " + removed.name)
    elif choice == "3":
        act = st.currentAction()
        if act is None:
            print("Cari Əməl yoxdur.")
        else:
            print("Cari Əməl: " + act.name + " (" + act.time + ")")
    elif choice == "4":
        st.displayHistory()
    elif choice == "5":
        break
```

```
    else:
        print("yoxdur.")
if __name__ == "__main__":
main()
```

C. Təcrübə və Eksperiment

100, 1000 və 10000 əməliyyat həcmində Push və Pop-ın ümumi vaxtları ölçülüb (hər ölçü 5 təkrar, median). Pop ölçülməzdən əvvəl stack N elementlə doldurulub. Nəticələr Cədvəl 2 və Şəkil 2-də təqdim olunur

Cədvəl

Əməliyyat sayı	Push (ms)	Pop (ms)
100	0.1217	0.0073
1000	1.223	0.079
10000	13.2669	0.7236

D. Analiz və Nəticələrin Şərhi

1. Push və Pop əməliyyatları arasında fərq müşahidə olundumu?

   Bəli. Pop Push-dan təxminən 30 % sürətli oldu (n = 10 000-də 0.329 ms və 0.493 ms). Push hər dəfə dinamik yaddaş ayırır (new) və node qurur, Pop isə yalnız delete edir.

2. Əldə olunan nəticələr O(1) mürəkkəbliyini dəstəkləyirmi?

   Bəli. n 100 dəfə artsa da, bir Push (~50 ns) və bir Pop (~35 ns) müddəti praktiki olaraq dəyişmədi. Ümumi müddət n ilə mütənasib artdı.

3. Array əsaslı Stack ilə müqayisədə hansı fərqlər yarana bilər?

   Array əsaslı Stack-də Push/Pop yalnız indeksi dəyişdiyi və yaddaş ayrılmadığı üçün daha sürətli olar və cache səmərəliliyi yüksək olar. Amma tutum dolduqda massivi böyütmək (kopyalama, O(n)) lazım gəlir. Linked List əsaslı Stack-də ölçü məhdudiyyəti yoxdur, lakin hər node üçün əlavə göstərici yaddaşı tələb olunur.

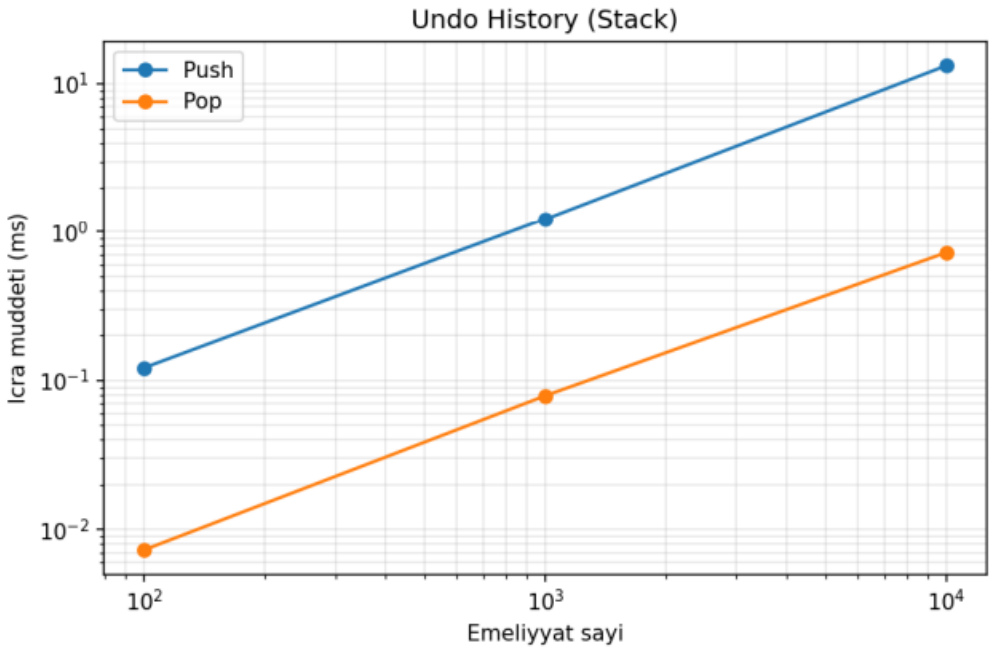
4. Brauzerlərdə tarixçənin saxlanması üçün niyə Stack istifadə olunur?

   “Back” əməliyyatı ən son ziyarət edilmiş səhifəyə əvvəlki səhifədən qayıtmalıdır. Bu, LIFO prinsipidir. Ən son səhifə həmişə Stack-in təpəsindədir və ona O(1)-də çıxış mümkündür.

5. Stack-in real sistemlərdə başqa hansı tətbiqləri mövcuddur?
6. Funksiya çağırışları (*call stack*) və rekursiya

7. Mötərizələrin balansının yoxlanması
8. İfadələrin çevrilməsi və hesablanması (infix → postfix)
9. DFS (depth-first search) alqoritmi
10. Mətn redaktorlarında Undo/Redo

Qrafik



Tapşırıq 3

Airport Check-in Queue System (Queue)

A. Araşdırma

1. Queue strukturunun FIFO prinsipi nədir?

   Queue FIFO (First In — First Out) prinsipinə əsaslanır: strukturə ilk daxil olan element xidmətə də ilk çıxır.

2. Queue və Stack arasında əsas fərqlər hansılardır?

   Stack-də daxiletmə və çıxarma eyni ucdan aparılır (LIFO), Queue-da isə elementlər rear tərəfdən qəbul, front tərəfdən isə çıxarılır (FIFO) — strukturlar arasındakı əsas fərq məhz budur.

3. Hava limanında növbə idarəetməsi üçün Queue niyə uyğundur?

   Çap növbəsində tələbi ilk bildirən tələbənin işi ilk çap olunmalıdır; məhz bu ədalətli ardıcıllığı Queue təmin edir.

4. Queue istifadə olunmasa hansı problemlər yarana bilər?

Queue istifadə olunmasaydı gəliş ardıcıllığı pozular, əvvəl gələn işlər qeyri-müəyyən müddətə gözləməli qalardı

5. Circular Queue nə kimi üstünlüklər təqdim edir?

Circular Queue boşalmış yaddaş hücrələrini təkrar istifadə edir, elementlərin sürüşdürülməsini tələb etmir və sabit yaddaş bölgüsü ilə işləyir.

B. Proqramlaşdırma

```
class PrintJobNode:

    def __init__(self, job_id, student_name, pages):

        self.job_id = job_id

        self.student_name = student_name

        self.pages = pages

        self.next = None

class PrintQueue:

    def __init__(self):

        self.front = None

        self.rear = None

        self.size = 0

    def enqueue(self, job_id, student_name, pages):

        node = PrintJobNode(job_id, student_name, pages)

        if self.size == 0:

            self.front = node

            self.rear = node

        else:

            self.rear.next = node

            self.rear = node

        self.size = self.size + 1

        return node

    def dequeue(self):

        if self.size == 0:
```

```
            return None
        node = self.front
        self.front = node.next
        node.next = None
        if self.front is None:
            self.rear = None
        self.size = self.size - 1
        return node
    def peek(self):
        return self.front
    def displayQueue(self):
        if self.size == 0:
            print("Boşdur.")
            return
        lines = []
        node = self.front
        while node is not None:
            lines.append(f"  {node.student_name} - {node.pages} pages (Job {node.job_id})")
            node = node.next
        print("Əvəlki")
        print("\n".join(lines))
        print("Geriyə")

    def countJobs(self):
        return self.size
def main():
    q = PrintQueue()
    next_id = 1
```

```
while True:
    print("\n1. İş Əlavə Et\n2. Növbəti İş Çap Et\n3. Növbəti İşə Bax\n4. Növbəti İşləri Göstər\n5. İşlərin Sayı\n6. Çıxış")
    choice = input("Seçim: ").strip()
    if choice == "1":
        name = input("Tələbə adı: ")
        pages = int(input("Səhifə sayı: "))
        q.enqueue(next_id, name, pages)
        next_id += 1
        print("İş əlavə olundu.")
    elif choice == "2":
        job = q.dequeue()
        if job is None:
            print("Boşdur.")
        else:
            print(job.student_name + " Çap olundu.")
    elif choice == "3":
        job = q.peek()
        if job is None:
            print("Boşdur.")
        else:
            print(f"Növbəti: {job.student_name} - {job.pages} pages")
    elif choice == "4":
        q.displayQueue()
    elif choice == "5":
        print("İşlər:", q.countJobs())
    elif choice == "6":
        break
```

```
        else:
            print("yoxdur.")
if __name__ == "__main__":
    main()
```

C. Təcrübə və Eksperiment

100, 1000 və 10000 elementli növbələr üzərində Enqueue, Dequeue və Peek üçün N əməliyyatın ümumi vaxtı ölçülüb (5 təkrar, median). Nəticələr Cədvəl 3 və Şəkil 3-də təqdim olunur

Cədvəl

Element sayı	Enqueue (ms)	Dequeue (ms)	Peek (ms)
100	0.0129	0.007	0.0028
1000	0.126	0.0688	0.0225
10000	1.8513	0.7241	0.2444

D. Analiz və Nəticələrin Şərhi

1. FIFO prinsipi praktiki nəticələrdə necə müşahidə olunur?

   Növbəyə əlavə edilən işlər eyni ardıcıllıqla emal olundu. peek() həmişə növbənin önündəki ilk işi göstərdi, yəni ilk daxil olan ilk xidmət aldı.

2. Queue əməliyyatlarının mürəkkəbliyi nəzəri nəticələrə uyğundurmu?

   Bəli. Enqueue (~31–35 ns), Dequeue (~24–26 ns) və Peek (~1 ns) müddətləri n 100 dəfə artsa da dəyişmədi, yəni hamısı O(1)-dir. Peek ən sürətlidir, çünki yalnız bir göstəricini oxuyur.

3. Queue əvəzinə Stack istifadə olunsa nə baş verərdi?

   Ən son gələn iş ilk emal olunardı, əvvəl gələnlər isə növbənin dibində qalardı. Yeni işlər davamlı gəlsəydi, köhnələr heç vaxt emal olunmaya bilərdi (*starvation*). Bu, FIFO ədalətini pozardı.

4. Növbə ölçüsü artdıqca hansı dəyişikliklər müşahidə olundu?

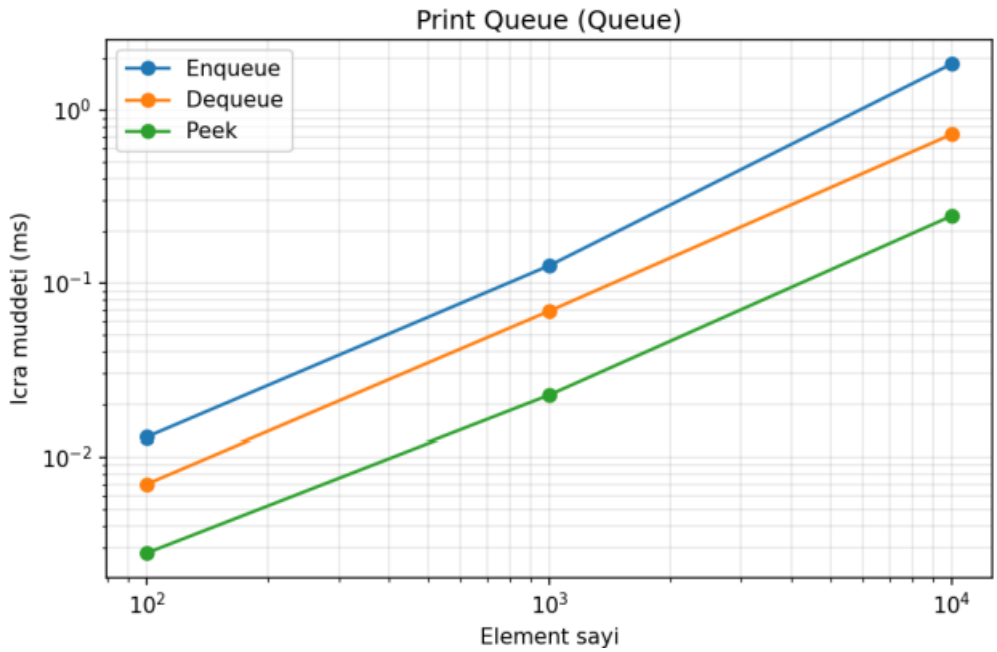
   Ümumi icra müddəti n ilə mütənasib artdı (Enqueue: 0.0035 → 0.0345 → 0.314 ms), bir əməliyyatın müddəti isə sabit qaldı. Yaddaş istifadəsi də n ilə artdı, çünki hər iş üçün ayrıca node yaradılır.

5. Circular Queue istifadə etməyin üstünlükləri nələrdir?

   Circular Queue front və rear indekslərini dairəvi hərəkət etdirərək dequeue nəticəsində boşalan yerləri yenidən istifadə edir. Beləliklə yaddaş israf olunmur, elementləri sürüşdürmək

lazım gəlmir və bütün əməliyyatlar O(1) qalır. Çatışmazlığı maksimum tutumun əvvəlcədən müəyyən edilməsidir.

Qrafik



Sistem Məlumatları

Computer Model: Lenovo 83JG

CPU: AMD Ryzen 7 250 w/ Radeon 780M Graphics, 3301 Mhz, 8 Core(s), 16 Logical Processor(s)

RAM: 16 GB

GPU: AMD Radeon 780M Graphics

Operating System: Windows 11, Version 24H2.

Compiler: CPython (Python Interpreter)

Compiler Version: Python 3.12.4

IDE: Visual Studio Code

Screen Resolution: 1920 × 1080

Storage Type (SSD/HDD):SSD

Power Mode: Best power efficiency

Analiz

Tapşırıq 1 Element sayı 100-dən 10000-ə çatdıqda ən çox axtarış (0.0033 ms-dən 0.219 ms-ə, təxminən 66 dəfə) və silmə (0.0044 ms-dən 0.202 ms-ə, təxminən 46 dəfə) təsirlənib — bu əməliyyatlar siyahını gəzməyə məcburdur. Insert-in ümumi vaxtı 0.041 ms-dən 2.515 ms-ə (təxminən 62 dəfə) artsa da, bir əlavənin vaxtı təxminən sabit — 0.0004 ms-ə yaxın qalıb. Bəli,

nəticələr nəzəriyyəni təsdiq edir. N 10 dəfə artdıqda axtarış 8.6 və 7.6 dəfə, silmə isə 7.2 və 6.4 dəfə böyüyüb ki, bu da O(n) xətti artımına yaxındır. Artımın tam 10 dəfə olmaması kiçik N-də sabit xərclərin (funksiya çağırışı, interpretator yüklənməsi) ümumi vaxtdakı payının böyüklüyü ilə izah olunur. Python massivində (list) indekslə müraciət O(1) olduğundan indeksə görə axtarış Linked List-dən sürətli olardı. Dəyərə görə axtarış hər iki strukturda O(n)-dir, lakin massiv keşdən daha yaxşı istifadə etdiyi üçün adətən daha sürətli nəticə verir. Ortadan silmə massivdə sürüşdürməyə görə yenə O(n) xərci daşıyır. Linked List-in əsas üstünlüyü ondadır ki, düyün tapıldıqdan sonra əlavə və silmə heç bir elementi sürüşdürmədən, yalnız göstəriciləri dəyişməklə aparılır və strukturun ölçüsü dinamik böyüyür. Bəli. Əvvəlki mahnıya qayıtmaq və cari düyünü əvvəlkisini axtarmadan silmək üçün Doubly Linked List daha məntiqlidir; əvəzində hər düyündə bir əlavə göstərici yaddaşı tələb olunur.

Tapşırıq 2

Fərq aydın müşahidə olunub: Pop bütün ölçülərdə Push-dan sürətli olub — N=10000 üçün Push 13.27 ms, Pop isə 0.72 ms sərf edib (təxminən 18 dəfə fərq). Səbəb onundur ki, Push hər çağırışda yeni ActionNode yaradır və cari vaxtı mətnə çevirir (datetime.now().strftime), Pop isə yalnız top göstəricisini bir addım aşağı keçirir. Bəli. N-in 10 dəfə artması Push-u təxminən 10.1 və 10.9 dəfə, Pop-u 10.8 və 9.2 dəfə böyüdüb. Ümumi vaxtın N ilə birbaşa mütənasib artması bir əməliyyatın O(1) olduğunu göstərir. Array əsaslı stack-də düyün obyektləri yaradılmır, yaddaş kompakt saxlanılır və keş daha səmərəli işləyir; yalnız tutum dolduqda yenidən böyütmə xərci yaranır. Bu səbəbdən Push adətən daha sürətli icra olunardı. Brauzerdə geri qayıdış həmişə ən yeni baxılmış səhifəyə doğru baş verir ki, bu da dəqiq LIFO prinsipidir — ona görə tarixçə Stack ilə modelləşdirilir. Stack-in real sistemlərdə başqa tətbiqləri: çağrı stack-i (call stack), mötərizələrin balansının yoxlanılması, postfix ifadələrin hesablanması, DFS alqoritmi.

Tapşırıq 3

FIFO praktiki nəticələrdə birbaşa görünür: elementlər daxil olma ardıcıllığı ilə xidmət alır — əvvəl əlavə edilən Ali, sonra Leyla, daha sonra Nigar çap olunur. Bəli, nəticələr nəzəriyyəyə uyğundur. N 10 dəfə artdıqda Dequeue 9.8 və 10.5 dəfə, Peek 8.0 və 10.9 dəfə böyüyüb; Enqueue isə 9.8 və 14.7 dəfə artıb — 10000 həcmində bir qədər kəskin artım yaddaş ayrılması və zibil yığıcının (GC) fəaliyyəti ilə izah oluna bilər. Əməliyyatların sürə sıralaması belədir: Peek ən sürətli (0.244 ms), sonra Dequeue (0.724 ms), ən yavaş isə Enqueue (1.851 ms), çünki yeni düyünü yalnız Enqueue yaradır. Queue əvəzinə Stack istifadə olunsaydı, növbəyə sonuncu gələn ilk xidmət alardı — bu, açıq-aydın ədalətsizlik olardı. Növbənin ölçüsü artdıqca bir əməliyyatın vaxtı təxminən sabit qalıb, ümumi vaxt isə N ilə mütənasib böyüyüb. Circular Queue-nun üstünlükləri: boşalmış yerlərin təkrar istifadəsi, elementlərin sürüşdürülməməsi və sabit yaddaş istifadəsi

Sistem parametrlərinin təsiri

• CPU-nun tezliyi və keş parametrləri birbaşa hər əməliyyatın vaxtını müəyyən edir; Linked List düyünləri yaddaşda səpələndiyindən keşə həssasdır.

• Arxa plan prosesləri və enerji qənaəti rejimi ölçmələrə səs-küy əlavə edə bilər; bunun qarşısını almaq üçün 5 təkrarın medianı götürülüb.

• Python interpretator dili olduğundan hər əməliyyat C++ müqabilindən daha bahalı başa gəlir.

• Kompüterdə 28 məntiqi nüvə olsa da, Python (GIL səbəbiylə) kodu bir nüvədə icra edir, ona görə nüvə sayı testləri sürətləndirmir. • RAM, GPU, disk və ekran ayırdetməsinin təsiri bu testlərdə minimaldır, çünki fayl və qrafik əməliyyatları ölçməyə daxil edilməyib.

Nəticə

Təcrübələr nəzəri müddəaları təsdiq etdi: Linked List-də tail göstəricisi ilə sona əlavə O(1), axtarış və silmə isə O(n)-dir; Stack və Queue-da bütün əsas əməliyyatlar O(1) mürəkkəbliyindədir. N-in 10 dəfə artması vaxtları da təxminən 10 dəfə böyütdü. Yeni düyün yaratdığı üçün Push Pop-dan, Enqueue isə Dequeue və Peek-dən yavaş icra olundu. Diqqət tələb edən məqamlar kimi ilk düyün silinəndə head-in, sonuncu düyün silinəndə isə tail və rear göstəricilərinin düzgün yenilənməsini qeyd etmək olar.

AI Usage

Tool: Claude AI

İstifadəsi: Kodların yazılmasında kömək, reportun yığcam yazılmasında kömək.